

INSTITUTO DE DESARROLLO ECONÓMICO E INNOVACIÓN

Año: 2017



PROGRAMA DE LA ASIGNATURA:

Laboratorio de Programación y Lenguajes (IF009)

CÓDIGO: IF009

AÑO DE UBICACIÓN EN EL PLAN DE ESTUDIOS:

3 año

FECHA ULTIMA REVISIÓN DE LA ASIGNATURA:

2017-04-07

CARRERA/S: Licenciatura en Sistemas V2,
Analista Universitario de Sistemas V2,

CARÁCTER: CUATRIMESTRAL (1ro)

TIPO: OBLIGATORIA

NIVEL: GRADO

MODALIDAD DEL DICTADO: PRESENCIAL

MODALIDAD PROMOCION DIRECTA: SI

CARGA HORARIA SEMANAL: 6 HS

CARGA HORARIA TOTAL: 90 HS

EQUIPO DOCENTE

Nombre y Apellido	Cargo	e-mail
Adiana Uricuolo	Profesor Asociado	aurciuolo@untdf.edu.ar
Matias Gel	Asistente Principal Interino	mgel@untdf.edu.ar
Federico González Brizzio	Asistente de Primera	fgonzalez@untdf.edu.ar

1. FUNDAMENTACION

Esta asignatura se enfoca en el desarrollo de aplicaciones concretas por parte de los alumnos, lo cual constituye un ejercicio similar a su futura actividad profesional, requiriendo por lo tanto de creatividad, para el planteo de la solución de problemas e integración de conocimientos para la implementación de dicha solución.

La materia se ubica en el primer cuatrimestre del tercer año, razón por la cual los alumnos ya han incorporado conocimientos relativos a diseño de algoritmos y estructuras de datos, programación OO, arquitectura y diseño de software, bases de datos y otros, sin haber tenido aún la oportunidad de integrar todos estos conceptos en el desarrollo de aplicaciones concretas. Si bien la correlatividad anterior está asignada con Algorítmica y Programación II, los contenidos mínimos expresan que se debe trabajar sobre la base de los conocimientos adquiridos los dos primeros años, razón por la cual se parte de considerar los mismos. No obstante ello, se brindan a nivel teórico los conceptos básicos introductorios a cada tema que, siendo necesarios para desarrollar aplicaciones, no estén contemplados en la correlatividad anterior.

La falta de integración del tratamiento de los conceptos mencionados, brindados a los alumnos durante los dos primeros años, provoca en el tercer año la necesidad de trabajar sobre ciertos conocimientos específicos para el desarrollo de aplicaciones concretas que se pretende encarar en la presente asignatura, como son, modelos de persistencia y su vínculo con los lenguajes de programación, el mapeo OO-relacional, testing unitario, el manejo de herramientas integradas de desarrollo y frameworks como ambientes de trabajo y ejecución para la construcción ágil de aplicaciones.

De acuerdo a lo expuesto, se hace necesaria la enseñanza de conceptos fundamentales para el desarrollo de aplicaciones concretas en diferentes lenguajes con soporte de ambientes integrados de programación que faciliten el desarrollo rápido de las mismas, los cuales incluyan una capa de

persistencia y facilidades para el mapeo OO-relacional de manera ágil y sencilla, así como para la construcción de interfaces de usuario y testing de aplicaciones.

2. OBJETIVOS

a) OBJETIVOS GENERALES

Se pretende que el alumno sea capaz de desarrollar en forma ágil y eficiente aplicaciones concretas, utilizando y profundizando los conocimientos de diseño, programación y lenguajes adquiridos previamente, mediante la utilización de herramientas y ambientes de programación y ejecución apropiados para la integración de los diferentes aspectos que conforman una aplicación.

b) OBJETIVOS ESPECIFICOS

Se pretende que mediante los contenidos dictados y su aplicación en el desarrollo de aplicaciones concretas, el alumno:

- Profundice sus conocimientos sobre programación y lenguajes para desarrollar aplicaciones de software de calidad.
- . Aprenda a utilizar herramientas de testing unitario.
- Amplíe sus conocimientos sobre el paso del diseño a la implementación de programas.
- Aprenda a resolver el problema de mapeo OO-relacional en bases de datos.
- Aprenda a utilizar herramientas que le permitan agilizar el desarrollo de aplicaciones.
- Comprenda las ventajas de desarrollar aplicaciones en el marco de un framework de aplicación.
- Conozca lenguajes y técnicas apropiadas para el desarrollo ágil de aplicaciones.
- Esté en condiciones de analizar y realizar comparaciones entre frameworks y lenguajes de programación desde el punto de vista de la implementación de aplicaciones.

3. CONDICIONES DE REGULARIDAD Y APROBACION DE LA ASIGNATURA

Para la aprobación del cursado de la asignatura se requiere:

- Cumplir con el 70% de asistencia a las clases prácticas.
- Aprobar la totalidad de los Trabajos Prácticos con Nota igual o superior a seis (6).
- Aprobar dos parciales prácticos bajo la forma de aplicaciones integradoras (a la mitad del cuatrimestre y al final del cuatrimestre) de los distintos conceptos analizados en la materia (o sus respectivos recuperatorios) con Nota igual o superior a cuatro (4).

La calificación final de la cursada es el promedio de las notas obtenidas en los exámenes parciales, los trabajos prácticos y las aplicaciones integradoras. En ningún caso la nota obtenida podrá ser inferior a 6.

Para la aprobación de la asignatura se deberá rendir un examen final,

Para la promoción de la asignatura, el alumno además de las condiciones anteriores, deberá proceder a:

- Aprobar los dos parciales prácticos en la primera instancia, con nota igual o superior a siete (7).
- Defensa del Desarrollo de las dos aplicaciones integradoras, en la cual debe obtener Nota igual o superior a siete (7).
- Presentación de un trabajo final en el cual debe obtener Nota igual o superior a siete (7)

4. CONTENIDOS DE LA ASIGNATURA

CONTENIDOS MÍNIMOS

Desarrollo de aplicaciones concretas en las que se integren conocimientos adquiridos en los dos

primeros años.

METODOLOGÍAS DE CLASES

Se dictarán clases teóricas relativas a los conceptos expuestos en el Programa analítico y en particular de su aplicación en el Lenguaje seleccionado para el desarrollo de la aplicación integradora, con un fuerte apoyo de ejemplos en ese y otros lenguajes conocidos por los alumnos, a los fines de que estén en condiciones de realizar comparaciones. Se realizarán presentaciones PowerPoint, así como la ejecución de programas ejemplos en los lenguajes seleccionados utilizando proyector y pantalla. Las herramientas IDE y los frameworks serán asimismo explicados mediante ejemplos desplegados en pantalla, desarrollados frente a los alumnos.

Durante las clases prácticas, se trabajará en base a una intensa práctica del funcionamiento y utilización de

herramientas que permita la agilización de aplicaciones, de los constructores provistos por los diferentes lenguajes a utilizar, mediante ejemplos y posterior ejercitación con el fin de comparar y evaluar las prestaciones de los mismos. Se realizará una intensa ejercitación y práctica de programación. Se utilizará un lenguaje específico como central para el desarrollo de aplicaciones en cada parte de la asignatura.

Se promoverá la lectura de artículos de interés y bibliografía específica, a los fines de generar hábitos de investigación y actualización de conocimientos, así como la búsqueda de información relativa a publicaciones sobre frameworks y herramientas en Internet, especialmente revistas científicas, proceedings

de congresos y seminarios. Todas las instancias implican autocorrección y/o corrección por parte del docente. Además de la atención permanente y la revisión de los trabajos prácticos, se brindarán clases de consulta en forma previa a los exámenes parciales y ante requerimientos puntuales, se dictarán clases adicionales para aclarar dudas.

El uso de bibliografía por parte del alumno: Los apuntes de cátedra se reducirán a tópicos generales de cada tema, incluyendo los conceptos básicos de los mismos en forma de hilo conductor. Se estima conveniente inducir a los estudiantes a trabajar con bibliografía que complemente los apuntes de clase, señalando en cada caso la más apropiada al tema en cuestión.

Se entiende que entre la teoría y la práctica debe haber una relación integrada, dado el intensivo uso de ejemplos y ejercicios que implica la aplicación de conceptos teóricos en el desarrollo de aplicaciones. De esta forma, las clases teóricas están complementadas con ejemplos y problemas prácticos apropiados para la mejor comprensión de la implementación de soluciones en diferentes lenguajes. Algunas clases prácticas serán precedidas por una introducción teórica para abordar el tema, ejemplos y ejercitación para el trabajo individual o grupal (según el caso); en otros casos, durante las clases prácticas los alumnos avanzarán en el desarrollo de aplicaciones, realizando pruebas, consultas y correcciones. Además de los ejercicios correspondientes a cada TP, se propondrán trabajos integradores en ambas partes de la asignatura (aplicaciones de escritorio y aplicaciones WEB) para que los alumnos realicen la experiencia de desarrollar aplicaciones concretas reales.

PROGRAMA ANALITICO

MÓDULO I: Diseño y construcción de aplicaciones en entornos de desarrollo ágil

El Modelo Ágil para construcción de aplicaciones. Herramientas de desarrollo ágil. Código auto documentado y herramientas de extracción de código. Construcción de aplicaciones en entornos de desarrollo integrado. Pautas y criterios para la transformación diseño-código. Patrones y desarrollo ágil. Desarrollo dirigido por pruebas. Conceptos. Objetivos. Importancia de la detección oportuna de errores. Tipos de Pruebas. Prueba de Unidad. Procedimiento para pruebas de unidad. Ventajas. Soporte en java para pruebas de unidad. Framework JUnit. Soporte de IDEs para utilización de JUnit.

MÓDULO II: Aplicaciones de escritorio. Interfaz de usuario

Construcción de aplicaciones de escritorio. Ejemplo en Java. Niveles de programación de interfaz. Tipos de Interfaz. Programación basada en eventos. Componentes y contenedores. Gestión de eventos. Layout de componentes. Validaciones. Creación de componentes visuales en diferentes entornos. Diseño de la interfaz. Soporte de interfaz de usuario brindado por frameworks de ámbito general y de aplicaciones web. Facilidades para la construcción de interfaces en los lenguajes de programación.

MÓDULO III: Aplicaciones de escritorio. Manejo de persistencia

Concepto de persistencia. Persistencia y su relación con los lenguajes de programación.

Soluciones al problema de la persistencia. Mapeo OO - Relacional. Comparación de tecnologías ORM (Object Relational Mapping). Persistencia en Java. JPA: Entidades, Gestor de persistencia, Mapeo Objeto-relación, Consultas JPQL. CRUD con JPA. Ejemplos. Persistencia en otros lenguajes de programación.

MÓDULO IV: Lenguajes y Frameworks para desarrollo de aplicaciones WEB

Características de aplicaciones WEB. Capas de una aplicación WEB. Cliente y Servidor.

Transferencia. Páginas WEB. Arquitectura de una aplicación WEB. Diseño de la aplicación.

Lenguajes dinámicos para desarrollo de aplicaciones WEB. Ejemplo: Lenguaje Groovy. El patrón MVC. Concepto de Framework como ambiente de trabajo y ejecución. Objetivos. Utilidad.

Ventajas. La importancia del uso de frameworks para desarrollo rápido. Capas de frameworks.

Soporte de programas. Bibliotecas. Frameworks para ámbitos específicos. Frameworks para entornos de programación WEB. El Modelo MVC en frameworks WEB. Ejemplos de Frameworks como ambientes para integrar y agilizar aplicaciones: El Framework Grails para el lenguaje Groovy.

MÓDULO V: Construcción de aplicaciones con lenguajes dinámicos y frameworks.

Uso de frameworks para el desarrollo de aplicaciones. Ejemplo: Framework Grails. Modelo de Dominio. Clases de dominio. Relaciones entre clases. Persistencia de clases: GORM. Métodos básicos CRUD. Validación. Queries. Finders dinámicos- Queries HQL. Hibernate criteria query Vistas. Vista y Modelo. GSP. Sintaxis básica. Tags. Forms y fields. Layout y templates.

Componentes de un layout. Ejemplos. Login. Controllers. Acciones. Acciones default. Retorno de un modelo. Rendering de respuesta. Interceptores. Data binding. Servicios en Grails. Creación, Llamada. Ejemplos. Transacciones, Scopes.

Modulo VI: Refactoring.

Análisis y modificación de código existente en busca de realizar mejoras en agregar nuevas funcionalidades. Malas prácticas. Malos Olores en el código. Casos de refactoring. Aplicar los casos en forma controlada. Utilización de herramientas de refactoring integradas al IDE.

5. RECURSOS NECESARIOS

- Proyector
- Pc

6. PROGRAMACIÓN SEMANAL

Semana	Unidad / Módulo	Descripción	Bibliografía
1	I	Entornos de desarrollo ágil. Utilización de IDEs. Pruebas Unitarias con JUnit. TP 1: Ejemplos y ejercicios con JUnit. El alumno escribe el comportamiento de una serie de clases y sus pruebas unitarias.	
2	I	Introducción a la persistencia de objetos. JDBC inicio con JPA TP Ejemplos de consultas con JPA.	

3	II	Persistencia en Java. JPA: Entidades, Mapeo de relaciones, Gestor de persistencia, CRUD con JPA TP Desarrollo de un modelo de dominio y una capa DAO. Presentación de aplicación integradora.	
4	III	Persistencia en Java. JPA: Consultas JPQL TP Desarrollo de un modelo de dominio y una capa DAO. Análisis de persistencia en aplicación integradora.	
5	III	Repaso de aplicaciones de escritorio con java TP Desarrollo de pruebas unitarias para la capa DAO realizada en el practico anterior. Desarrollo de la capa DAO para la aplicación integradora	
6	III	Ejemplo de aplicación completa con persistencia usando JPA en el entorno de un IDE. TP: Finalización aplicación en JAVA con JPA en el entorno de un IDE. Consultas avances aplicación integradora.	
7	I II III	Finalización aplicación integradora en JAVA. Presentación de tps completos de aplicaciones de escritorio Parcial práctico N° 1	
8	IV	Conceptos de aplicaciones WEB. MVC. Lenguajes dinámicos y Frameworks para aplicaciones WEB. Lenguaje Groovy. Framework Grails. TP: Aplicaciones en Groovy Presentación pautas de Aplicación web integradora Parcial recuperatorio N° 1	
9	V	Conceptos avanzados del lenguaje Groovy. Ejecución de programas con Scaffolding estático y dinámico en Grails. TP Aplicaciones en Lenguaje Grovy	
10	V	Modelo de Dominio en Grails. Clases de dominio. Relaciones entre clases. TP: Modelo de dominio en Grails. Aplicación en Lenguaje Groovy. Implementación modelo dominio aplicación web integradora	
11	V	Vistas en Grails. Vista y Modelo. GSP. Sintaxis básica. Tags. Forms y fields. Layout y templates. Ejemplos: Login. TP: Construcción de interfaz de Aplicación en Groovy. Diseño Interfaz aplicación web integradora	
12	V	Controllors y Servicios en Grails.Acciones. TP: Uso de controllors en la aplicación groovy-grails. Control de Avances en la aplicación integradora.	
13	V	Persistencia en Grails. GORM. Métodos básicos CRUD. Validación. Queries. Finders dinámicos- Queries HQL. TP: CRUD y Queries en aplicacioens Groovy-Grails.	
14	VI	Refactoring. Detectar malas prácticas. Aplicar cambios en forma controlada . Refactoring. Detectar malas prácticas. Aplicar cambios en forma controlada . TP: Ejemplos de refactoring.	
15	-	Ejemplo de aplicación completa en Groovy-Grails TP: Parcial práctico N° 2	
16	-	Presentación de aplicación integradora y de aplicaciones de trabajaos práctico completos. Parcial recuperatorio N° 2	
17	-	Reunión de cátedra para elaboración de Informe final.	

7. BIBLIOGRAFIA DE LA ASIGNATURA

Autor	Año	Título	Capítulo/s	Lugar de la Edición	Editor / Sitio Web
Gamma Erich, Helm Richard , Johnson Ralph , Vlissi	1997	Patrones de diseño, Elementos de software orientado a objetos reutilizable	1-6		Pearson Educacion
Freeman E. & Freeman	2004	Head First Design Patterns	1-14		E. O'Reilly
Zukowski, John	2005	The Definitive Guide to Java Swing	1-22		apress
Mike Keith, Merrick Schincariol	2009	Pro JPA 2: Mastering the Java Persistence API (Expert's Voice in Java)	1-11		apress
James Sutherland, Doug Clarke	2013	Java Persistence	1-11		wikibooks.org
Koenig D.	2011	Groovy in Action 2a Ed.	1-9		Manning Publication
Glen Smith	2014	Grails in Action	1-4		Manning Publications
Rocher G., Brown J.	2009	The definitive guide to Grails.	1-12		Apress
Kent Beck	2002	Test Driven Development: By Example.	Sección 3		Addison-Wesley
Martin Fowler, Kent Beck	1999	Refactoring: Improving the Design of Existing Code	1-13		Addison-Wesley Professional

Firma del docente-investigador responsable

VISADO		
COORDINADOR DE LA CARRERA	DIRECTOR DEL INSTITUTO	SECRETARIO ACADEMICO UNTDF
Fecha :	Fecha :	

Este programa de estudio tiene una validez de hasta tres años o hasta que otro programa lo reemplace en ese periodo